```
!pip install -q openai requests python-dotenv pandas numpy scikit-learn scipy statsmodels matplotlib seaborn gigachat
!apt-get update -qq && apt-get install -y -qq pciutils zstd
```

```
W: Skipping acquire of configured file 'main/source/Sources' as repository 'https://r2u.stat.illinois.edu/ubuntu noble InRel
```

```
import subprocess
import time
import os

# Установка Ollama
!curl -fsSL https://ollama.com/install.sh | sh

env = os.environ.copy()
env["LD_LIBRARY_PATH"] = "/usr/lib64-nvidia:" + env.get("LD_LIBRARY_PATH", "")
# Запуск сервера в фоне
proc = subprocess.Popen(
    ["ollama", "serve"],
    stdout=subprocess.DEVNULL,
    stderr=subprocess.DEVNULL,
)
time.sleep(5)
```

```
>>> Cleaning up old version at /usr/local/lib/ollama
>>> Installing ollama to /usr/local
>>> Downloading ollama-linux-amd64.tar.zst
######################################################################## 100.0%
>>> Adding ollama user to video group...
>>> Adding current user to ollama group...
>>> Creating ollama systemd service...
WARNING: systemd is not running
>>> NVIDIA GPU installed.
>>> The Ollama API is now available at 127.0.0.1:11434.
>>> Install complete. Run "ollama" from the command line.
```

```
!nvidia-smi
```

```
Thu Oct 29 18:48:25 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   69C    P0             43W /   70W |       0MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

```
# Скачивание модели. Выбирайте ОДНУ из трёх опций в зависимости от того,
# что вы хотите сравнивать. См. критические замечания ниже.
# Опция A: qwen3:4b (text-only, 4B) — быстрая, мало VRAM
!ollama pull qwen3:4b-instruct
LOCAL_MODEL_NAME = "qwen3:4b-instruct"

# Опция B: qwen2.5:7b (text-only, 7B) — лучше качество, больше VRAM
# !ollama pull qwen2.5:7b
# LOCAL_MODEL_NAME = "qwen2.5:7b"

# Опция C: qwen2.5vl:7b (мультимодальная, ~8B) — ближе к qwen3-vl-8b из LM Studio
# !ollama pull qwen2.5vl:7b
# LOCAL_MODEL_NAME = "qwen2.5vl:7b"

# Проверка
!curl -s http://localhost:11434/api/tags | head -c 500
```

```
{"models":[{"name":"qwen3:4b-instruct","model":"qwen3:4b-instruct","modified_at":"2026-10-29T18:48:26.404410029Z","size":249
```

```
# === Ячейка 4 (исправленная): файлы и настройка только под local_qwen ===
import os, json, shutil
```

```
from google.colab import files

os.makedirs("logs", exist_ok=True)
os.makedirs("results", exist_ok=True)
os.makedirs("data", exist_ok=True)

print("Выберите: config.py, clients.py, bank.py, prompts.py, type_e_data.py, evaluate.py, "
      "ai4i2020.csv, tasks.jsonl (тот же банк, что локально), type_e_split.json (из локальной папки results). "
      ".env — по желанию (ключи для local_qwen не нужны).")
uploaded = files.upload()

for fname in ("ai4i2020.csv", "test_tasks.jsonl", "tasks.jsonl"):
    if os.path.exists(fname):
        shutil.move(fname, os.path.join("data", fname))

import config

# Drive обязателен: логи здесь общие с ноутбуком
from google.colab import drive
drive.mount("/content/drive")
config.LOGS_DIR = "/content/drive/MyDrive/lr3/logs"
config.RESULTS_DIR = "/content/drive/MyDrive/lr3/results"
os.makedirs(config.LOGS_DIR, exist_ok=True)
os.makedirs(config.RESULTS_DIR, exist_ok=True)
print("Логи и результаты пишутся на Google Drive")

# Банк — тот же файл, что в локальном config.py
BANK_FILE = "data/tasks.jsonl"   # или "data/test_tasks.jsonl", если итоговый банк ещё не готов
assert os.path.exists(BANK_FILE), f"Банк {BANK_FILE} не найден — проверьте загрузку файлов"
config.BANK_PATH = BANK_FILE

# Локальная модель — Ollama в этом же runtime
config.MODELS["local_qwen"]["base_url"] = "http://localhost:11434/v1"
config.MODELS["local_qwen"]["model_name"] = "qwen3:4b-instruct"  # сверьте с `!ollama list`

# Разбиение типа E: берём файл, созданный локально, чтобы индексы строк совпадали
split_local = "type_e_split.json"
split_drive = os.path.join(config.RESULTS_DIR, "type_e_split.json")
if os.path.exists(split_local):
    if os.path.exists(split_drive):
        a = json.load(open(split_local, encoding="utf-8"))
        b = json.load(open(split_drive, encoding="utf-8"))
        if a == b:
            print("type_e_split.json совпадает с уже лежащим на Drive")
        else:
            print("ВНИМАНИЕ: загруженный и дисковый сплиты различаются! "
                  "Используется версия на Drive — на неё уже могут ссылаться записи в логах.")
    else:
        shutil.copy(split_local, split_drive)
        print("type_e_split.json скопирован на Drive")
else:
    print("ПРЕДУПРЕЖДЕНИЕ: type_e_split.json не загружен; Colab создаст свой "
          "(детерминирован, seed=42, но файл лучше перенести явно)")

import clients
clients.VERIFY = True  # gigachat в Colab не вызывается; патч на всякий случай

print("Банк существует:", os.path.exists(config.BANK_PATH))
```

```
Выберите: config.py, clients.py, bank.py, prompts.py, type_e_data.py, evaluate.py, ai4i2020.csv, tasks.jsonl (тот же банк, ч
Выбрать файлы Файл не выбран  Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving .env to .env (2)
Saving ai4i2020.csv to ai4i2020.csv
Saving bank.py to bank (2).py
Saving clients.py to clients (2).py
Saving combined_ca.pem to combined_ca (2).pem
Saving config.py to config (2).py
Saving evaluate.py to evaluate (2).py
Saving prompts.py to prompts (2).py
Saving run_bank.py to run_bank (2).py
Saving run_type_e.py to run_type_e (2).py
Saving tasks.jsonl to tasks.jsonl
Saving type_e_data.py to type_e_data (2).py
Saving type_e_split.json to type_e_split (1).json
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=Tr
Логи и результаты пишутся на Google Drive
type_e_split.json совпадает с уже лежащим на Drive
Банк существует: True
```

```
import os, subprocess, time, urllib.request

_refs = []  # держать файловые дескрипторы логов живыми
```

```
def ollama_ready(timeout: float = 3.0) -> bool:
    try:
        with urllib.request.urlopen("http://127.0.0.1:11434/api/tags", timeout=timeout) as r:
            return r.status == 200
    except Exception:
        return False

def start_ollama() -> None:
    """Убивает остатки сервера и запускает заново, лог — в /content/ollama.log."""
    subprocess.run(["pkill", "-f", "ollama serve"],
                   stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
    time.sleep(2)
    log = open("/content/ollama.log", "ab", buffering=0)
    _refs.append(log)
    env = os.environ.copy()
    env["OLLAMA_KEEP_ALIVE"] = "-1"        # модель не выгружается из VRAM между запросами
    env["OLLAMA_HOST"] = "127.0.0.1:11434" # явно IPv4, без неоднозначности ::1
    p = subprocess.Popen(["ollama", "serve"], stdout=log, stderr=log, env=env)
    for _ in range(30):
        if ollama_ready():
            print("Ollama отвечает на 127.0.0.1:11434")
            return
        if p.poll() is not None:
            raise RuntimeError(
                f"ollama serve завершился с кодом {p.returncode}; "
                "причина: !tail -n 50 /content/ollama.log")
        time.sleep(1)
    raise RuntimeError("Ollama не поднялась за 30 с; причина: !tail -n 50 /content/ollama.log")

if ollama_ready():
    print("Ollama уже работает — перезапуск не нужен")
else:
    start_ollama()

# Прогрев: первый запрос после старта грузит модель в VRAM (~90 с).
# Делаем это здесь, чтобы прогон не начинался с долгого запроса.
from openai import OpenAI
_c = OpenAI(base_url="http://127.0.0.1:11434/v1", api_key="ollama")
_t0 = time.time()
_r = _c.chat.completions.create(
    model="qwen3:4b-instruct",   # сверьте с выводом `!ollama list`
    messages=[{"role": "user", "content": "Скажи: ок"}],
    max_tokens=8, temperature=0.0,
)
print(f"Модель прогрета за {time.time() - _t0:.1f} с:", _r.choices[0].message.content)
```

```
Ollama отвечает на 127.0.0.1:11434
Модель прогрета за 3.2 с: Ок! 😊 Что у тебя
```

```
# === Ячейка 5 (Colab): только local_qwen, прогресс, автоперезапуск Ollama ===
import json, time, os
import config
from config import MODELS, BANK_PATH, LOGS_DIR
from bank import load_bank
from prompts import build_prompt, build_type_e_prompt
from type_e_data import load_ai4i, make_split, examples_for, row_text, LABEL
from clients import ask
from openai import APIConnectionError

TEMP = 0.0
MODEL_KEYS = ["local_qwen"]   # остальные модели выполняются локально

class LocalModelDead(RuntimeError):
    pass

_streak = {mk: 0 for mk in MODEL_KEYS}

def ask_robust(model_key, prompt, task_id, task_type, meta=None):
    try:
        r = ask(model_key, prompt, temperature=TEMP,
                task_id=task_id, task_type=task_type, meta=meta)
        _streak[model_key] = 0
        return r
    except APIConnectionError:
        _streak[model_key] += 1
        if _streak[model_key] >= 3:
            raise LocalModelDead(
                "Ollama не отвечает после трёх перезапусков; "
                "причина: !tail -n 50 /content/ollama.log")
        print(f"  {task_id}: соединение с Ollama потеряно, перезапуск сервера...")
        start_ollama()  # из ячейки B
```

```
        return ask(model_key, prompt, temperature=TEMP,
                   task_id=task_id, task_type=task_type, meta=meta)

def completed_pairs() -> set:
    done = set()
    for fname in os.listdir(LOGS_DIR):
        if not fname.endswith(".jsonl"):
            continue
        mk = fname[:-len(".jsonl")]
        with open(os.path.join(LOGS_DIR, fname), encoding="utf-8") as f:
            for line in f:
                line = line.strip()
                if not line:
                    continue
                e = json.loads(line)
                if e.get("task_id") is not None:
                    done.add((mk, e["task_id"], e.get("n_shots")))
    return done

def fmt_eta(seconds: float) -> str:
    seconds = int(max(seconds, 0))
    return f"{seconds // 60}м {seconds % 60:02d}с"
```

```
# Дымовой тест — только локальная модель
try:
    ans, tok, dt, raw = ask_robust("local_qwen", "1",1,'A')
    print(f"local_qwen: OK {dt:.1f}s, {tok} токенов -> {ans.strip().splitlines()[-1][:70]}")
except Exception as e:
    print(f"local_qwen: ОШИБКА {type(e).__name__}: {e}")
```

```
local_qwen: OK 2.5s, 287 токенов -> УВЕРЕННОСТЬ: 0
```

```
done = completed_pairs()
print(f"Уже выполнено записей: {len(done)}")

# --- Часть 1: банк задач ---
tasks = load_bank(BANK_PATH)
for model_key in MODEL_KEYS:
    n_done, n_err = 0, 0
    for task in tasks:
        ttype = task["тип"].upper()
        if ttype == "E":
            continue
        if (model_key, task["id"], None) in done:
            continue
        prompt = build_prompt(task, lang="ru")
        try:
            ask_robust(model_key, prompt, task["id"], ttype)
            n_done += 1
        except LocalModelDead:
            raise
        except Exception as e:
            n_err += 1
            print(f"{model_key} {task['id']} ОШИБКА {type(e).__name__}: {e}")
        time.sleep(0.1)
    print(f"[банк] {model_key}: новых {n_done}, ошибок {n_err}")

# --- Часть 2: тип E с прогрессом ---
df = load_ai4i()
split = make_split(df, save_path=os.path.join(config.RESULTS_DIR, "type_e_split.json"))
rows = split["test_idx"]
for model_key in MODEL_KEYS:
    for n in (0, 5, 20):
        examples = examples_for(df, split, n)
        todo = [i for i in rows if (model_key, f"E_n{n}_{i}", n) not in done]
        done_n = len(rows) - len(todo)
        if not todo:
            print(f"[тип E] {model_key} n={n}: все {len(rows)} уже выполнены")
            continue
        print(f"[тип E] {model_key} n={n}: выполнено {done_n}/{len(rows)}, осталось {len(todo)}")
        n_ok, n_err = 0, 0
        t0 = time.perf_counter()
        for k, i in enumerate(todo, 1):
            tid = f"E_n{n}_{i}"
            prompt = build_type_e_prompt(row_text(df.loc[i]), examples)
            try:
                ask_robust(model_key, prompt, tid, "E",
                           meta={"n_shots": n, "row_idx": int(i),
                                 "y_true": int(df.loc[i, LABEL])})
                n_ok += 1
            except LocalModelDead:
                raise
```

```
            except Exception as e:
                n_err += 1
                print(f"{model_key} {tid} ОШИБКА {type(e).__name__}: {e}")
            time.sleep(0.1)
            if k % 10 == 0 or k == len(todo):
                el = time.perf_counter() - t0
                per = el / k
                print(f"  [{model_key} n={n}] {k}/{len(todo)} "
                      f"(всего {done_n + k}/{len(rows)}), "
                      f"{per:.1f} с/запрос, прошло {fmt_eta(el)}, "
                      f"осталось ~{fmt_eta(per * (len(todo) - k))}, ошибок {n_err}")
        print(f"[тип E] {model_key} n={n}: новых {n_ok}, ошибок {n_err}")

done = completed_pairs()
print(f"Всего записей с task_id в логах: {len(done)}")
```

```
Уже выполнено записей: 148
[банк] local_qwen: новых 0, ошибок 0
[тип E] local_qwen n=0: все 100 уже выполнены
[тип E] local_qwen n=5: выполнено 8/100, осталось 92
  E_n5_8702: соединение с Ollama потеряно, перезапуск сервера...
Ollama отвечает на 127.0.0.1:11434
  [local_qwen n=5] 10/92 (всего 18/100), 43.1 с/запрос, прошло 7м 10с, осталось ~58м 53с, ошибок 0
  [local_qwen n=5] 20/92 (всего 28/100), 40.7 с/запрос, прошло 13м 34с, осталось ~48м 52с, ошибок 0
  [local_qwen n=5] 30/92 (всего 38/100), 41.6 с/запрос, прошло 20м 48с, осталось ~43м 01с, ошибок 0
  [local_qwen n=5] 40/92 (всего 48/100), 41.9 с/запрос, прошло 27м 54с, осталось ~36м 16с, ошибок 0
  [local_qwen n=5] 50/92 (всего 58/100), 42.1 с/запрос, прошло 35м 03с, осталось ~29м 27с, ошибок 0
  [local_qwen n=5] 60/92 (всего 68/100), 42.1 с/запрос, прошло 42м 03с, осталось ~22м 26с, ошибок 0
  [local_qwen n=5] 70/92 (всего 78/100), 41.6 с/запрос, прошло 48м 30с, осталось ~15м 14с, ошибок 0
  [local_qwen n=5] 80/92 (всего 88/100), 41.3 с/запрос, прошло 55м 06с, осталось ~8м 15с, ошибок 0
  [local_qwen n=5] 90/92 (всего 98/100), 41.3 с/запрос, прошло 61м 56с, осталось ~1м 22с, ошибок 0
  [local_qwen n=5] 92/92 (всего 100/100), 41.3 с/запрос, прошло 63м 16с, осталось ~0м 00с, ошибок 0
[тип E] local_qwen n=5: новых 92, ошибок 0
[тип E] local_qwen n=20: выполнено 0/100, осталось 100
  [local_qwen n=20] 10/100 (всего 10/100), 44.1 с/запрос, прошло 7м 21с, осталось ~66м 12с, ошибок 0
  [local_qwen n=20] 20/100 (всего 20/100), 44.4 с/запрос, прошло 14м 47с, осталось ~59м 08с, ошибок 0
  [local_qwen n=20] 30/100 (всего 30/100), 43.3 с/запрос, прошло 21м 38с, осталось ~50м 29с, ошибок 0
  [local_qwen n=20] 40/100 (всего 40/100), 43.6 с/запрос, прошло 29м 03с, осталось ~43м 34с, ошибок 0
  [local_qwen n=20] 50/100 (всего 50/100), 43.8 с/запрос, прошло 36м 29с, осталось ~36м 29с, ошибок 0
  [local_qwen n=20] 60/100 (всего 60/100), 43.9 с/запрос, прошло 43м 56с, осталось ~29м 17с, ошибок 0
  [local_qwen n=20] 70/100 (всего 70/100), 43.8 с/запрос, прошло 51м 04с, осталось ~21м 53с, ошибок 0
  [local_qwen n=20] 80/100 (всего 80/100), 44.0 с/запрос, прошло 58м 43с, осталось ~14м 40с, ошибок 0
  [local_qwen n=20] 90/100 (всего 90/100), 44.3 с/запрос, прошло 66м 30с, осталось ~7м 23с, ошибок 0
  [local_qwen n=20] 100/100 (всего 100/100), 44.4 с/запрос, прошло 74м 00с, осталось ~0м 00с, ошибок 0
[тип E] local_qwen n=20: новых 100, ошибок 0
Всего записей с task_id в логах: 340
```